

codeHive

Trees

HIERARCHICAL & BRANCHING LOGIC

DSA Mastery Series: Part 24

codeHive

1. Beyond Linear Structures

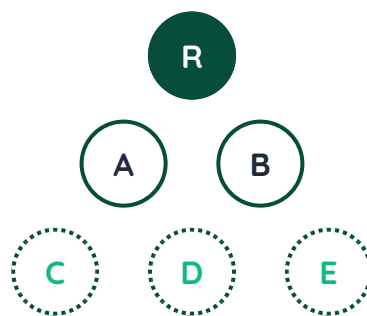
Unlike Arrays or Linked Lists, which are **Linear** (one element follows another), a **Tree** is a non-linear structure. It allows a single node to branch out to multiple "next" elements, creating a hierarchical map similar to a family tree or a file directory.

Why utilize Trees?

- **Organization:** Perfect for nested data like HTML (DOM) or folders.
- **Efficiency:** Dramatically faster searching than linear scans.
- **Routing:** Essential for networking and AI pathfinding.

2. Tree Anatomy & Rules

To master trees, you must speak the language of "nodes" and "edges."



Root: The absolute top node (Node R).

Child: A node branching from a parent.

Leaf: Nodes with no children (The end points).

Edge: The link connecting two nodes.

Parent: A node that has links to children.

Height: Max edges from root to a leaf.

Golden Rule: Every node (except the root) can have only **one** parent, but can have **many** children.

codeHive

3. Taxonomy of Trees

Depending on the rules of the branches, we categorize trees into specific types:

1. Binary Trees

The standard foundation. In a binary tree, each node can have a maximum of **two** children (Left and Right).

2. Binary Search Trees (BST)

A sorted binary tree. The value of the **Left Child** is always smaller than the parent, and the **Right Child** is always larger.

3. AVL Trees

A **Self-Balancing** BST. It automatically rotates its branches during insertion to ensure the height difference between sub-trees is never more than 1, keeping search times at $O(\log n)$.

4. Practice Arena

Scenario: You are looking at a tree where nodes C, D, E, and G have no links to any other nodes further down.

Question: What is the formal name for these specific nodes?

Answer: Leaf Nodes (or Leaves)

© 2026 codeHive Academy • Mastering Tree Hierarchies